



UD7_Práctica4_Triggers.

Índice

UD7_Práctica4_Triggers.....	1
Documentación e información.....	2
Triggers.....	3
Ejercicios sobre triggers.....	3



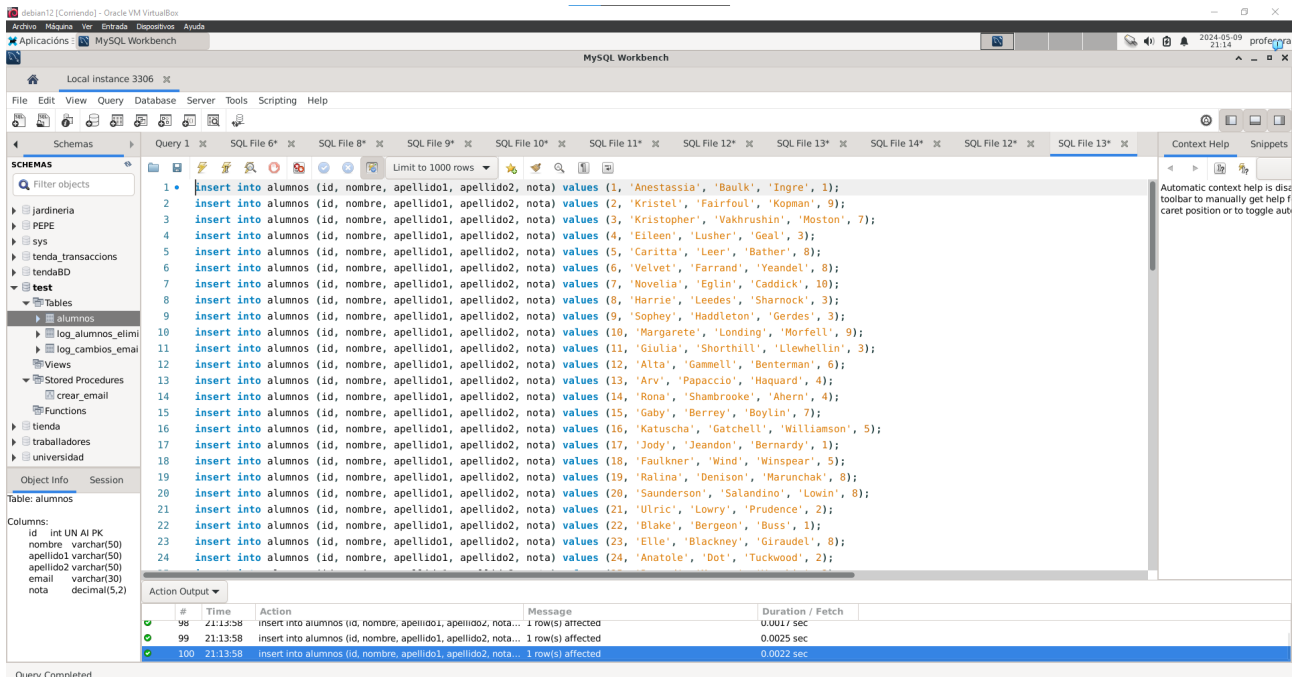
Documentación e información.

A base de datos de jardinería téndela dispoñible no curso da aula virtual, a carón do bloque de exercicios.

Moitos dos exercicios fanse sobre as mesmas BD, polo que podedes aproveitar o que teñades feito duns exercicios para outros.

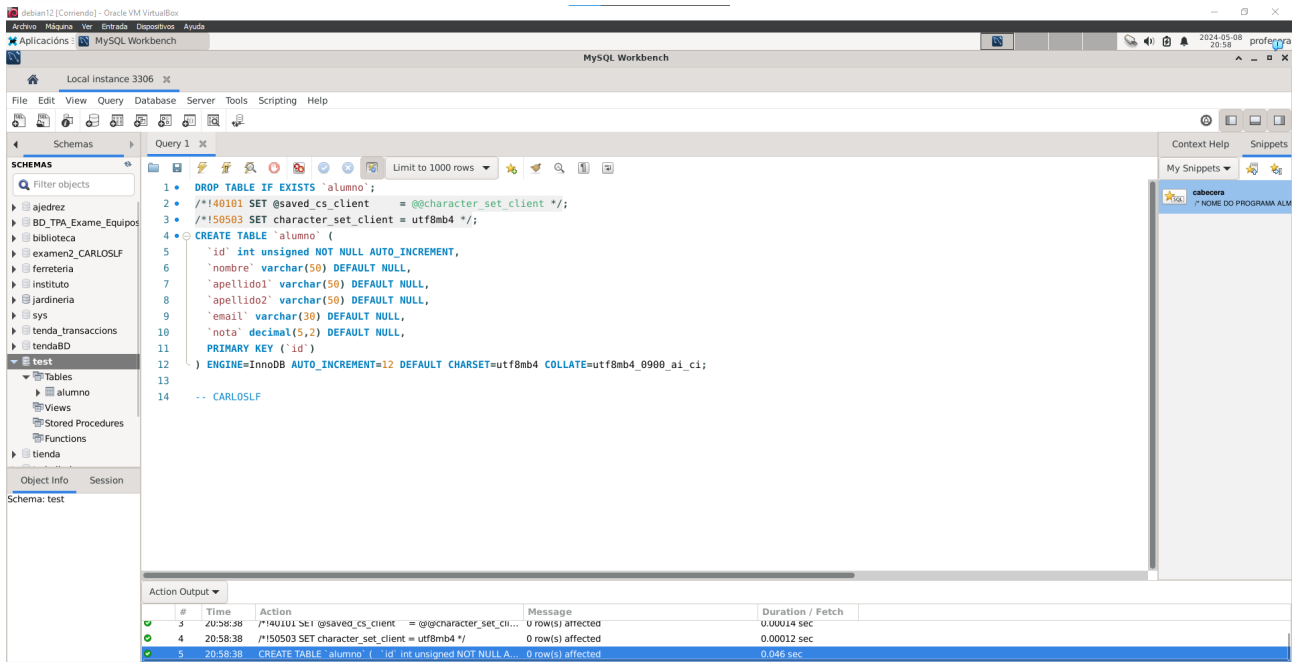


Triggers.



Exercicios sobre triggers.

1. Crea unha base de datos chamada **test** que conteña unha **táboa chamada alumnos** coas seguintes columnas.
 - ide (enteiro sen signo)
 - nome (cadea de caracteres)
 - apelido1 (cadea de caracteres)
 - apelido2 (cadea de caracteres)
 - nota (número real)



Unha vez creada a táboa **escriba dous triggers** coas seguintes características:

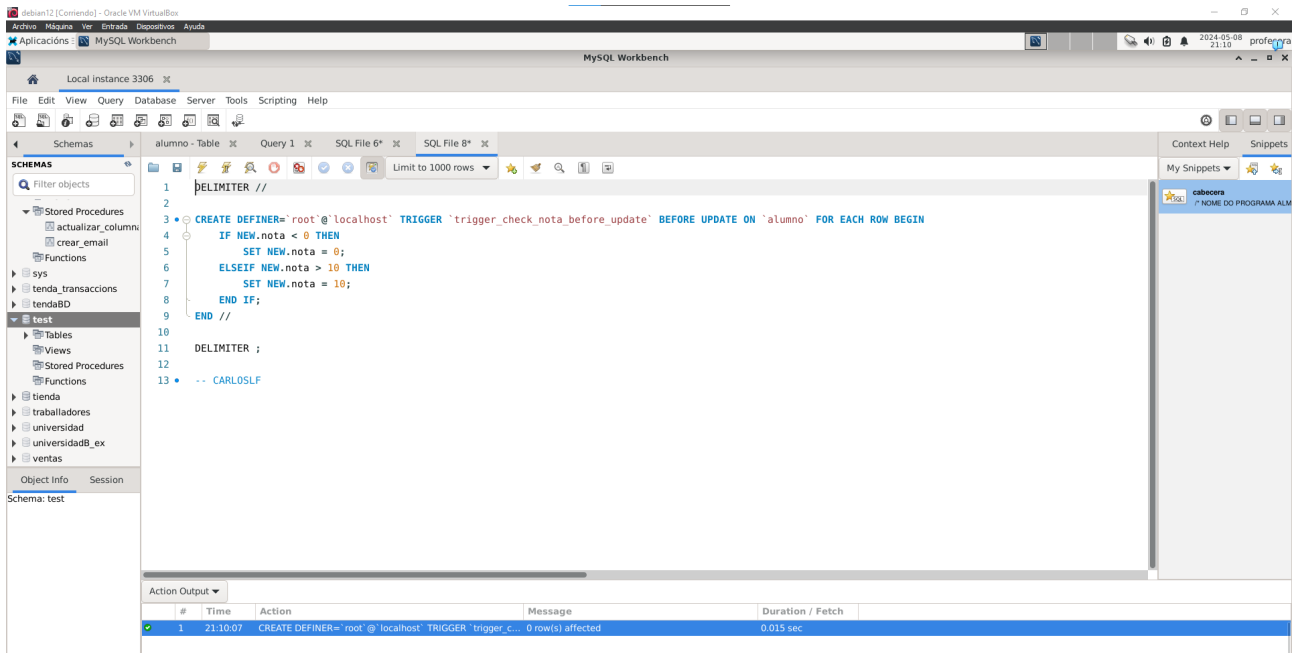
- Trigger 1: `trigger_check_nota_before_insert`
 - Execútase sobre a táboa `alumnos`.
 - Execútase antes dunha operación de inserción.
 - Se o novo valor da nota que se quere inserir é negativo, se garda como 0.
 - Se o novo valor da nota que se quere inserir é maior que 10, se garda como 10.



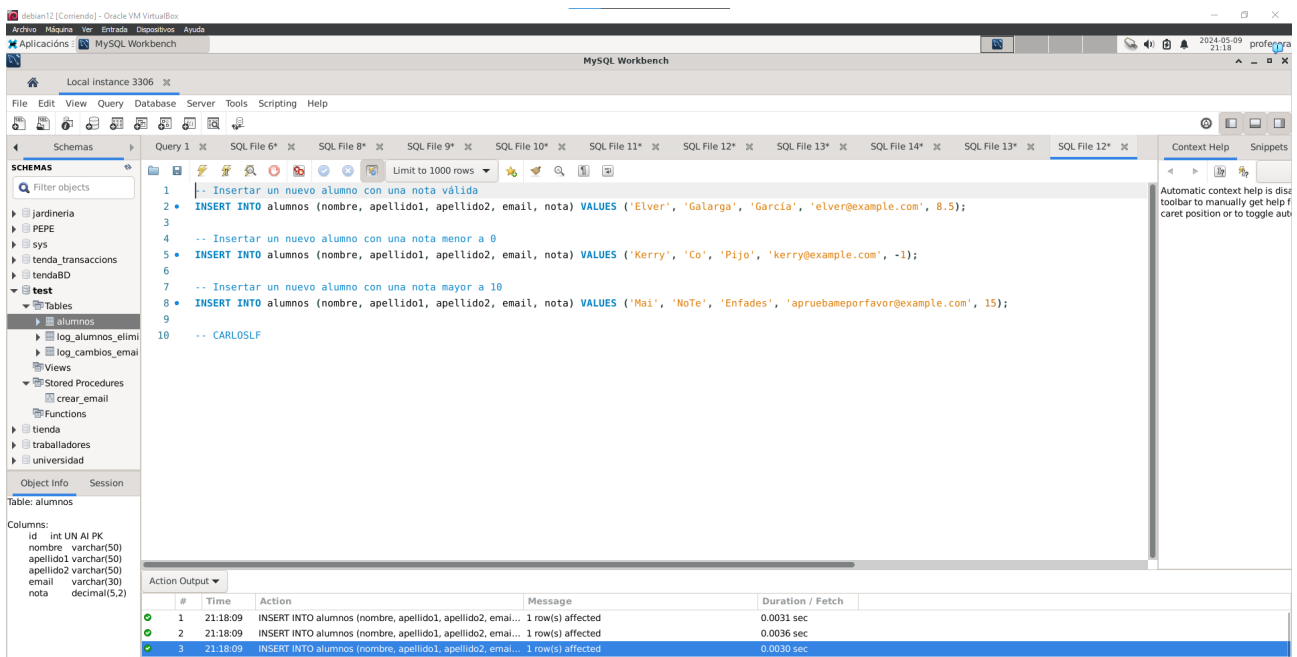
```
1 DELIMITER //
2
3 CREATE DEFINER='root'@'localhost' TRIGGER `trigger_check_nota_before_insert` BEFORE INSERT ON `alumno` FOR EACH ROW BEGIN
4 IF NEW.nota < 0 THEN
5 SET NEW.nota = 0;
6 ELSEIF NEW.nota > 10 THEN
7 SET NEW.nota = 10;
8 END IF;
9 END //
10
11 DELIMITER ;
12
13 -- CARLOSLE
```

#	Time	Action	Message	Duration / Fetch
1	21:08:08	CREATE DEFINER='root'@'localhost' TRIGGER `trigger_c...	0 row(s) affected	0.0082 sec

- Trigger2 : trigger_check_nota_before_update
 - Execútase sobre a táboa alumnos.
 - Execútase antes dunha operación de actualización.
 - Se o novo valor da nota que se quere actualizar é negativo, se garda como 0.
 - Se o novo valor da nota que se quere actualizar é maior que 10, se garda como 10.



Unha vez creados os triggers escriba varias sentenzas de inserción e actualización sobre a táboa alumnos e verifica que os triggers están a executarse correctamente.





MySQL Workbench interface showing a query execution result. The query is: `SELECT * FROM test.alumnos;`

The result grid displays the following data:

#	id	nombre	apellido1	apellido2	email	nota
90	90	Morissa	Spurdon	Baudi		2.00
91	91	Georgiea...	Gahan	Beller		2.00
92	92	Brynn	Dineen	Banfield		5.00
93	93	Ainslie	Wahner	Bradno...		5.00
94	94	Ruthe	Ney	Orton		5.00
95	95	Bella	Akitt	Brother...		5.00
96	96	Darbie	Courtman	Merigeau		4.00
97	97	Kalley	von Groll...	Boleyn		4.00
98	98	Karia	Grauit	Primrose		6.00
99	99	Katine	Chastaing	Belhouse		9.00
100	100	Prince	Boud	Erwin		5.00
101	101	Elver	Galarga	Garcia	elv...	8.50
102	102	Kerry	Co	Pijo	ker...	0.00
103	103	Mai	NoTe	Enfades	apr...	10.00

The Action Output shows the execution details:

#	Time	Action	Message	Duration / Fetch
1	21:19:16	SELECT * FROM test.alumnos LIMIT 0, 1000	103 row(s) returned	0.00070 sec / 0.000...

MySQL Workbench interface showing a query execution result. The query is: `UPDATE alumnos SET nota = 9 WHERE id = 1;`

The result grid displays the following data:

#	id	nombre	apellido1	apellido2	email	nota
90	90	Morissa	Spurdon	Baudi		2.00
91	91	Georgiea...	Gahan	Beller		2.00
92	92	Brynn	Dineen	Banfield		5.00
93	93	Ainslie	Wahner	Bradno...		5.00
94	94	Ruthe	Ney	Orton		5.00
95	95	Bella	Akitt	Brother...		5.00
96	96	Darbie	Courtman	Merigeau		4.00
97	97	Kalley	von Groll...	Boleyn		4.00
98	98	Karia	Grauit	Primrose		6.00
99	99	Katine	Chastaing	Belhouse		9.00
100	100	Prince	Boud	Erwin		5.00
101	101	Elver	Galarga	Garcia	elv...	8.50
102	102	Kerry	Co	Pijo	ker...	0.00
103	103	Mai	NoTe	Enfades	apr...	10.00

The Action Output shows the execution details:

#	Time	Action	Message	Duration / Fetch
1	21:19:57	UPDATE alumnos SET nota = 9 WHERE id = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.0043 sec
2	21:19:57	UPDATE alumnos SET nota = -2 WHERE id = 2	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.0034 sec
3	21:19:57	UPDATE alumnos SET nota = 12 WHERE id = 3	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.0030 sec



MySQL Workbench interface showing a database named 'test' with a table named 'alumnos'. The table has columns: id, nombre, apellido1, apellido2, email, and nota. The table contains 12 rows of data. The 'Result Grid' shows the output of a query: 'SELECT * FROM test.alumnos;'. The 'Action Output' pane shows the execution of three UPDATE statements and the final SELECT statement.

#	id	nombre	apellido1	apellido2	email	nota
1	1	Anastasia	Bauk	Ingre		9.00
2	2	Kristel	Fairboul	Kopman		0.00
3	3	Kristopher	Vakhrushin	Moston		10.00
4	4	Eileen	Lusher	Geal		3.00
5	5	Caritta	Leer	Bather		8.00
6	6	Velvet	Farrand	Yeandel		8.00
7	7	Novelia	Eglin	Caddick		10.00
8	8	Harrie	Leedes	Sharnock		3.00
9	9	Sophey	Haddleton	Gerdas		3.00
10	10	Margarete	Londing	Morrell		9.00
11	11	Gailla	Shorthill	Lienwhelin		3.00
12	12	Alta	Gammell	Benter		6.00

Table: alumnos

Columns:

- id int UN AI PK
- nombre varchar(50)
- apellido1 varchar(50)
- apellido2 varchar(50)
- email varchar(30)
- nota decimal(5,2)

Action Output

#	Time	Action	Message	Duration / Fetch
1	21:19:57	UPDATE alumnos SET nota = 9 WHERE id = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.0043 sec
2	21:19:57	UPDATE alumnos SET nota = -2 WHERE id = 2	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.0034 sec
3	21:19:57	UPDATE alumnos SET nota = 12 WHERE id = 3	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.0030 sec
4	21:20:36	SELECT * FROM test.alumnos LIMIT 0.1000	103 row(s) returned	0.00066 sec / 0.000...

2. Crea unha **base de datos chamada test** que conteña unha **táboa** chamada **alumnos** coas seguintes columnas:

- ide (enteiro sen signo)
- nome (cadea de caracteres)
- apelido1 (cadea de caracteres)
- apelido2 (cadea de caracteres)
- email (cadea de caracteres)

Escriba un **procedemento chamado crear_email** que dados os parámetros de entrada: nome, apelido1, apelido2 e dominio, cree unha dirección de email e devólvaas como saída.

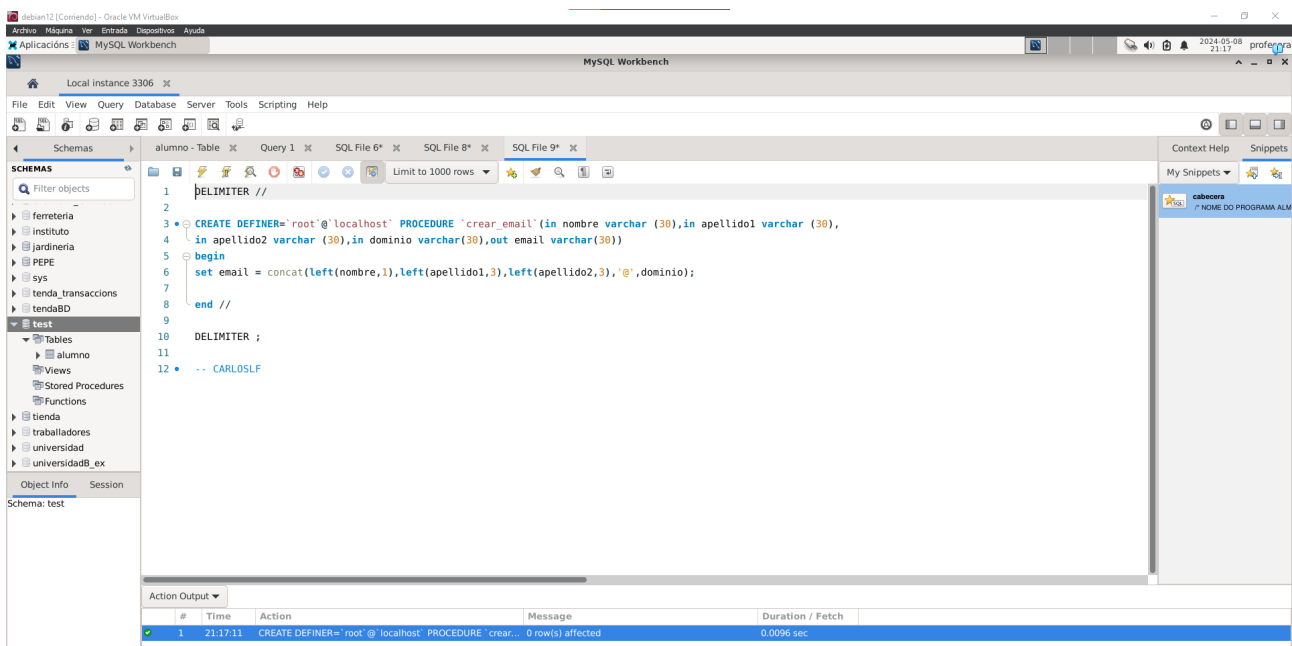
- Procedemento: crear_email
- Entrada:
 - nome (cadea de caracteres)
 - apelido1 (cadea de caracteres)
 - apelido2 (cadea de caracteres)
 - dominio (cadea de caracteres)



- Saída: - email (cadea de caracteres)

devolva unha dirección de correo electrónico co seguinte formato:

1. O primeiro carácter do parámetro nome.
2. Os tres primeiros caracteres do parámetro apelido1.
3. Os tres primeiros caracteres do parámetro apelido2.
4. O carácter @.
5. O dominio pasado como parámetro.



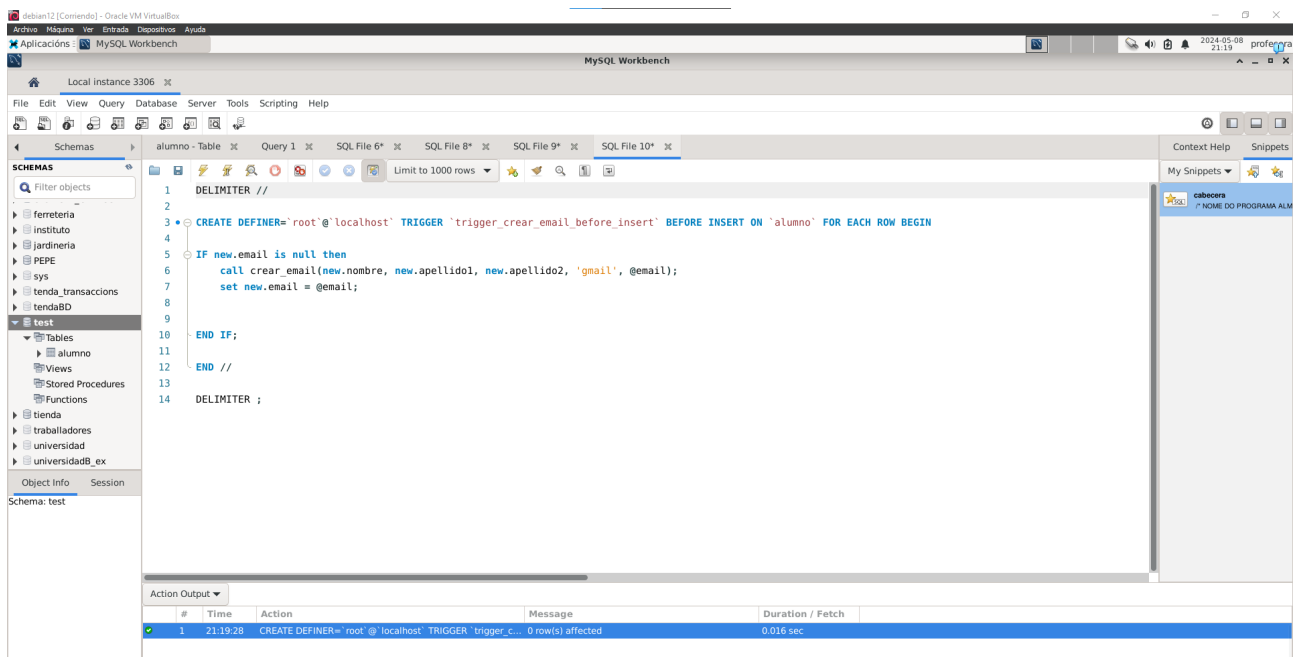
3. Unha vez creada a táboa escriba un **trigger** coas seguintes características:

- Trigger: trigger_crear_email_before_insert
- Execútase sobre a táboa alumnos.
- Execútase antes dunha operación de inserción.
- Se o novo valor do email que se quere inserir é NULL, entón crearáselle automaticamente unha dirección de email e inserírase na táboa.



- Se o novo valor do email non é NULL gardarase na táboa o valor do email.

Nota: Para crear a nova dirección de email deberase facer uso do procedemento `crear_email`.



4. Modifica o exercicio anterior e **engade un novo trigger** que as seguintes características:

- Trigger: `trigger_gardar_email_after_update`
- Execútase sobre a táboa `alumnos`.
- Execútase despois dunha operación de actualización.
- Cada vez que un alumno modifique a súa dirección de email deberase inserir un novo rexistro nunha táboa chamada `log_cambios_email`.

A **táboa `log_cambios_email`** contén os seguintes campos:

- `ide`: clave primaria (enteiro autonumérico)
- `ide_alumno`: ide do alumno (enteiro)
- `data_hora`: marca de tempo co instante do cambio (data e hora)
- `old_email`: valor anterior do email (cadea de caracteres)



- new_email: novo valor co que se actualizou

The screenshot shows the MySQL Workbench interface. The 'Schemas' pane on the left shows the 'test' database selected. The 'Query' pane in the center contains the following SQL code:

```
1 CREATE TABLE `log_cambios_email` (  
2   `id` int NOT NULL AUTO_INCREMENT,  
3   `id_alumno` int unsigned DEFAULT NULL,  
4   `fecha_hora` datetime DEFAULT NULL,  
5   `old_email` varchar(30) DEFAULT NULL,  
6   `new_email` varchar(30) DEFAULT NULL,  
7   PRIMARY KEY (`id`),  
8   KEY `id_alumno` (`id_alumno`),  
9   CONSTRAINT `log_cambios_email_ibfk_1` FOREIGN KEY (`id_alumno`) REFERENCES `alumno` (`id`)  
10 ) ENGINE=InnoDB AUTO_INCREMENT=8 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci  
11  
12 -- CARLOSLF
```

The 'Action Output' pane at the bottom shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	21:26:59	CREATE TABLE `log_cambios_email` (`id` int NOT NULL...	0 rows(s) affected	0.057 sec

The screenshot shows the MySQL Workbench interface. The 'Schemas' pane on the left shows the 'test' database selected. The 'Query' pane in the center contains the following SQL code:

```
1 DELIMITER //  
2  
3 CREATE DEFINER='root'@'localhost' TRIGGER `trigger_guardar_email_after_update` AFTER UPDATE ON `alumno` FOR EACH ROW BEGIN  
4 IF old.email not LIKE new.email THEN  
5   insert into log_cambios_email (id_alumno,fecha_hora,old_email,new_email) values (old.id,now(),old.email,new_email);  
6 END IF;  
7 END //  
8  
9 DELIMITER ;  
10  
11 -- CARLOSLF
```

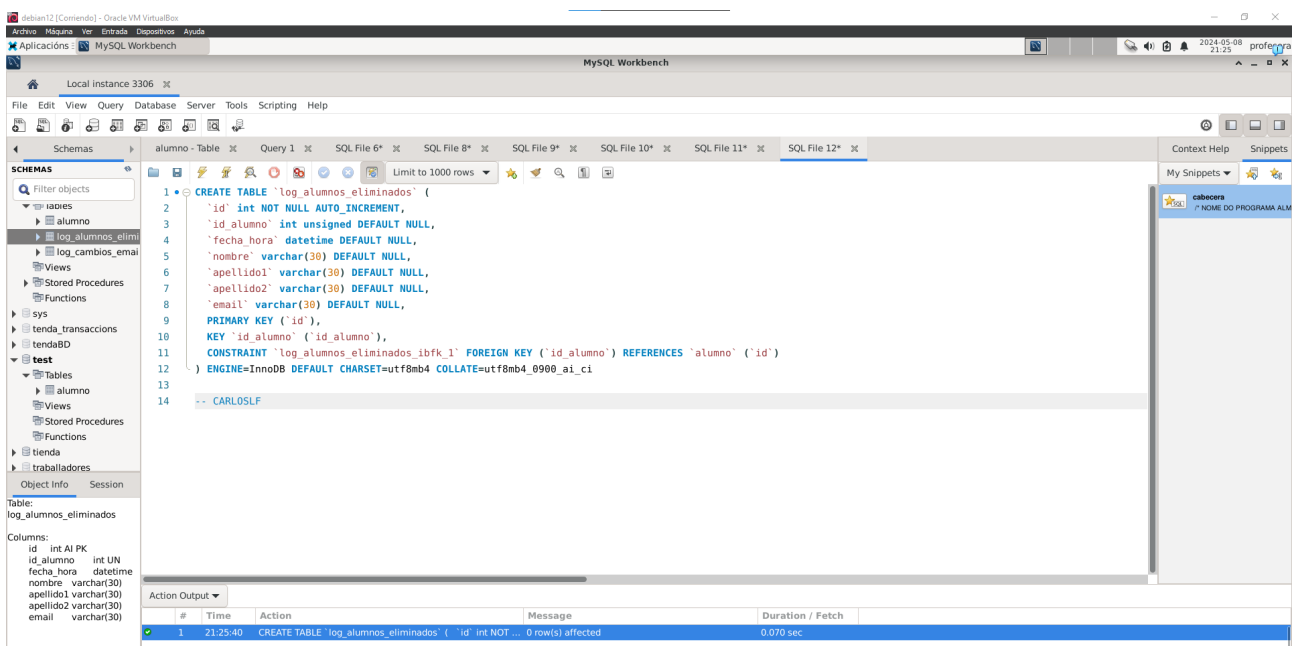
The 'Action Output' pane at the bottom shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	21:20:54	CREATE DEFINER='root'@'localhost' TRIGGER `trigger_g...	0 rows(s) affected	0.016 sec



5. Modifica o exercicio anterior e engade un **novo trigger** que teña as seguintes características:

- Trigger: trigger_gardar_alumnos_eliminados
- Execútase sobre a táboa alumnos.
- Execútase despois dunha operación de borrado.
- Cada vez que se elimine un alumno da táboa alumnos deberase inserir un novo rexistro nunha táboa chamada log_alumnos_eliminados.



A táboa log_alumnos_eliminados contén os seguintes campos:

- ide: clave primaria (enteiro autonumérico)
- ide_alumno: ide do alumno (enteiro)
- data_hora: marca de tempo co instante do cambio (data e hora)
- nome: nome do alumno eliminado (cadea de caracteres)
- apelido1: primeiro apelido do alumno eliminado (cadea de caracteres)
- apelido2: segundo apelido do alumno eliminado (cadea de caracteres)



- email: email do alumno eliminado (cadea de caracteres)

The screenshot shows the MySQL Workbench interface. The main editor contains the following SQL code:

```
1 DELIMITER //
2
3 CREATE DEFINER='root'@'localhost' TRIGGER 'trigger_log_alumnos_eliminados' AFTER DELETE ON 'alumno' FOR EACH ROW BEGIN
4     INSERT INTO log_alumnos_eliminados (id_alumno, fecha_hora, nombre, apellid1, apellid2, email)
5     VALUES (old.id, now(), old.nombre, old.apellid1, old.apellid2, old.email);
6 END //
7
8 DELIMITER ;
9
10 -- CARLOS LF
```

The left sidebar shows the 'Schemas' tree with the following structure:

- CHEMAS
 - Filter objects
 - raores
 - alumno
 - log_alumnos_elimi
 - log_cambios_email
 - Views
 - Stored Procedures
 - Functions
 - sys
 - tenda_transaccions
 - tendaBD
 - test
 - Tables
 - alumno
 - Views
 - Stored Procedures
 - Functions
 - tienda
 - trabajadores
 - Object Info
 - Session

The bottom status bar shows the execution result:

#	Time	Action	Message	Duration / Fetch
1	21:28:26	CREATE DEFINER='root'@'localhost' TRIGGER 'trigger_log_alumnos_eliminados' AFTER DELETE ON 'alumno' FOR EACH ROW BEGIN	0 row(s) affected	0.021 sec